

Esercizio 1 (35 punti)

Si consideri il tipo ricorsivo `bool_expr` definito come segue:

```
type bool_expr = Var of string
                | Not of bool_expr
                | And of bool_expr * bool_expr
                | Or of bool_expr * bool_expr
```

che consente di esprimere espressioni booleane con notazione prefissa. Per esempio `And (Or (Var "a", Var "b"), And (Var "a", Var "b"))` equivale all'espressione booleana in notazione infissa $(a \vee b) \wedge (a \wedge b)$.

1. (5 punti) Si definisca una funzione `eval12 : string -> bool -> string -> bool -> bool_expr -> bool` tale che `eval12 var1 b1 var2 b2 expr` restituisca il valore di verità dell'espressione booleana `expr` dove:

- le uniche variabili che *possono* occorrere in `expr` sono `var1` e `var2`;
- se in `expr` occorre una variabile diversa da `var1` e `var2`, viene restituito un messaggio di errore;
- i valori di verità delle variabili `var1` e `var2` sono rispettivamente `b1` e `b2`.

2. (5 punti) Utilizzando la funzione `eval12` del punto precedente, si definisca una funzione `table2 : string -> string -> bool_expr -> (bool * bool * bool) list` tale che `table2 a b expr` restituisca la tavola di verità dell'espressione booleana `expr` nelle due variabili `a` e `b`. Si assume che le variabili contenute in `expr` siano solo quelle espresse nei primi due argomenti di `table2`. Il valore restituito deve essere una lista di triple `(value_of_var1, value_of_var2, value_of_expr)`, per qualsiasi valore di verità delle variabili nei primi due argomenti di `table2`. Per esempio:

```
table2 "a" "b" (And (Var "a", Or (Var "a", Var "b"))) =
  [(true, true, true); (true, false, true); (false, true, false); (false, false, false)]
```

che corrisponde alla tavola di verità

<i>a</i>	<i>b</i>	$a \wedge (a \vee b)$
t	t	t
t	f	t
f	t	f
f	f	f

3. (5 punti) Si definisca una funzione `eval : (string * bool) list -> bool_expr -> bool` tale che `eval val_vars expr` restituisca il valore di verità dell'espressione booleana `expr`, dati i valori di verità delle sue variabili definiti nella lista associativa `val_vars`. Si tratta di una generalizzazione di `eval12`.
4. (20 punti) Si definisca una funzione `table : string list -> bool_expr -> ((string * bool) list * bool) list` che generalizzi il punto precedente in modo tale che l'espressione booleana da valutare possa contenere un qualsiasi numero di variabili. Pertanto `table lst expr` restituisce la tavola di verità della espressione booleana `expr` che può contenere solo variabili nella lista `lst`. Per esempio:

```
table ["a"; "b"] (And (Var "a", Or (Var "a", Var "b"))) =
  [([("a", true); ("b", true)], true); ([("a", true); ("b", false)], true);
   ([("a", false); ("b", true)], false); ([("a", false); ("b", false)], false)]
```

Suggerimento: Utilizzando la funzione `eval` del punto precedente, si definisca una funzione `table_maker : (string * bool) list -> string list -> bool_expr -> ((string * bool) list * bool) list` tale che `table_maker dict vars expr` restituisca la lista dei valori di verità dell'espressione booleana `expr` associati alla lista associativa `dict` e a ogni possibile combinazione dei valori di verità delle variabili contenute nella lista di stringhe `vars` (si proceda ricorsivamente su `vars`). Per esempio:

```
table_maker [("a", true)] ["b"] (And (Var "a", Or (Var "a", Var "b"))) =
  [([("a", true); ("b", true)], true); ([("a", true); ("b", false)], true)]
```